



MODUL 13 · UPLOAD FILE & OPTIMASI GAMBAR

# *Dasar Upload File via Server Action & Route Handler*

FormData + File API di server. Server Action untuk form upload, Route Handler untuk API.



U P L O A D   F I L E   &   N E X T / I M A G E

# File Upload — Bagaimana Server Menerima File

Browser kirim multipart/form-data. Server terima sebagai FormData. File = Blob dengan nama dan tipe.

```
// — Client: Form HTML —
<form action={uploadAction} encType="multipart/form-data">
  <input
    type="file"
    name="avatar"
    accept="image/*" // filter di browser
    multiple={false} // satu file saja
  />
  <button type="submit">Upload</button>
</form>

// — Server Action: terima File dari FormData —
('use server')
export async function uploadAction(formData: FormData) {
  const file = formData.get('avatar')

  // Type guard
  if (!(file instanceof File)) {
    return { error: 'File tidak ditemukan' }
  }
  if (file.size === 0) {
    return { error: 'File kosong' }
  }

  // File properties tersedia:
  // file.name    → "photo.jpg"
  // file.type    → "image/jpeg"
  // file.size    → 102400 (bytes)
  // await file.arrayBuffer() → ArrayBuffer (binary)

  // Proses selanjutnya: validasi, simpan ke storage
  return { received: true, name: file.name }
}
```

## File Properties

file.name: string

Nama file asli dari user.

JANGAN gunakan sebagai path!

Generate nama sendiri.

file.type: string

MIME type:

"image/jpeg", "image/png",

"image/webp", "application/pdf"

file.size: number (bytes)

1KB = 1024 bytes

1MB = 1024 \* 1024 bytes

await file.arrayBuffer():

Binary data untuk diproses  
atau dikirim ke cloud.

await file.text():

Untuk text/CSV files.

U P L O A D   F I L E   &   N E X T   /   I M A G E

# Upload ke Local Storage (Dev Only!)

public/uploads untuk development. Tidak persistent di Vercel production — gunakan cloud!

```
"use server"
import { writeFile, mkdir } from "fs/promises"
import path                  from "path"
import { nanoid }            from "nanoid"

export async function uploadLocal(formData: FormData) {
  const file = formData.get("file") as File | null
  if (!file || file.size === 0) return { error: "File tidak ada" }

  // 1. Konversi File → Buffer
  const buffer = Buffer.from(await file.arrayBuffer())

  // 2. Generate nama unik — JANGAN pakai file.name langsung!
  //   Security risk: path traversal, collision, special chars
  const ext      = path.extname(file.name).toLowerCase() // ".jpg"
  const filename = `${nanoid()}${ext}`                  //
"xK8mN3p.jpg"
  // 3. Pastikan direktori ada
  const uploadDir = path.join(process.cwd(), "public", "uploads")
  await mkdir(uploadDir, { recursive: true })

  // 4. Tulis file
  await writeFile(path.join(uploadDir, filename), buffer)

  // 5. Return public URL
  return { url: `/uploads/${filename}` }
}

// ⚠ WARNING:
// Tambahkan public/uploads ke .gitignore!
// Di Vercel: filesystem ephemeral → file HILANG setelah deploy!
// Gunakan cloud storage (Bab 2) untuk production.
```

bun add nanoid — generate ID unik. Kenapa bukan file.name?  
path traversal: '../../../etc/passwd',  
collision: dua 'photo.jpg' saling override,  
special chars: spasi/unicode bermasalah.

U P L O A D   F I L E   &   N E X T / I M A G E

# Upload via Route Handler — Untuk Public API

Gunakan Route Handler jika upload dipanggil dari mobile app atau client dengan fetch().

```
import { NextRequest, NextResponse } from "next/server"
import { auth } from "@lib/auth"
import { headers } from "next/headers"

export async function POST(req: NextRequest) {
  // 1. Auth — wajib!
  const session = await auth.api.getSession({ headers: await headers() })
  if (!session) return NextResponse.json({ error: "Unauthorized" }, { status: 401 })

  // 2. Parse FormData
  const formData = await req.formData()
  const file = formData.get("file") as File | null
  if (!file || file.size === 0) {
    return NextResponse.json({ error: "No file" }, { status: 400 })
  }

  // 3. Validasi cepat
  const MAX = 5 * 1024 * 1024
  const TYPES = ["image/jpeg", "image/png", "image/webp", "image/gif"]
  if (!TYPES.includes(file.type)) {
    return NextResponse.json({ error: "Tipe tidak diizinkan" }, { status: 422 })
  }
  if (file.size > MAX) {
    return NextResponse.json({ error: "Maks 5MB" }, { status: 422 })
  }

  // 4. Upload ke cloud (Bab 2)
  // const result = await uploadToCloudinary(file)
  // return NextResponse.json({ url: result.secure_url })
  return NextResponse.json({ message: "Ready for cloud upload" })
}
```

# Evaluasi Pemahaman

Jawab sebelum lanjut.

**Q1 — Bagaimana cara mendapatkan binary data dari File di Server Action?**

A) `file.data`

B) `Buffer.from(await file.arrayBuffer())`

C) `file.readAsBuffer()`

**Q2 — Mengapa TIDAK boleh menggunakan `file.name` sebagai nama file di server?**

A) Nama terlalu panjang

B) Security risk: path traversal, collision, special characters

C) Server tidak bisa baca nama file

**Q3 — Mengapa local storage (public/uploads) TIDAK cocok untuk production Vercel?**

A) Terlalu lambat

B) Filesystem Vercel ephemeral — file hilang setelah deployment baru

C) Vercel tidak support fs module



# Tugas Mandiri

Implementasikan file upload dasar ke local storage untuk development.

## 01 Upload Form

Buat UploadForm dengan `<input type='file' accept='image/*'>`.  
Gunakan Server Action `uploadLocal`. Tampilkan preview gambar setelah upload.

## 02 Server Action

Buat `uploadLocal` Server Action: terima file → nanoid filename → `writeFile` ke `public/uploads/` → return `{ url: '/uploads/...' }`.

## 03 Test Upload

Upload gambar → cek `public/uploads/` ada file-nya → render dengan `<img src={url}>` → verifikasi tampil di browser.

## 04 .gitignore

Tambahkan `/public/uploads/` ke `.gitignore`. Buat `/public/uploads/.gitkeep` agar folder tetap ada di repo tapi isi-nya tidak di-commit.



`bun add nanoid` — untuk generate unique ID. `nanoid()` menghasilkan string 21 karakter random yang URL-safe.



# Yang Sudah Kita Pelajari

- ✓ `formData.get('file')` as `File` → dapatkan file dari Server Action atau Route Handler.
- ✓ File properties: `.name`, `.type`, `.size`, `.arrayBuffer()`. Gunakan `arrayBuffer()` untuk binary.
- ✓ Generate nama file sendiri (nanoid + ext) — JANGAN gunakan `file.name` asli (security!).
- ✓ Local storage: HANYA dev. `Filesystem Vercel ephemeral` — file hilang setelah deploy.
- ✓ Server Action untuk form upload (rekomendasi). Route Handler untuk public API.

→ Selanjutnya: **Bab 2 — Upload ke Cloud Storage (Cloudinary)**

